InterviewAce

Test Case Specification Document

Backend Module · FastAPI / AI Scoring Engine

Document Type	Test Case Specification
Platform	InterviewAce
Scope	Backend — FastAPI / AI Scoring Engine
Prepared By	Backend Engineer
Date	May 2026
Version	1.0

1. Purpose

This document defines the backend test cases for the InterviewAce interview-prep platform. It covers the core use cases implemented in the FastAPI backend, including user authentication, AI-powered answer scoring, feedback generation, progress tracking, and question bank management. Each test case is traceable to a specific use case documented in the Use Case Analysis.

2. Test Cases

TC-01	JWT User Authentication	PASS
Use Case: UC-AUTH — Authenticate User		*Priority: Critical*
Objective	*Verify that the FastAPI auth layer registers new users with bcrypt-hashed passwords, issues a valid JWT upon correct credentials, and rejects invalid or missing credentials.*	
Preconditions	FastAPI server is running. A registered test user (email: test@interviewace.com, password: SecurePass123!) exists in the test DB.	

#	Action / Input	Expected Result
1	POST /auth/register with a new valid email and password.	HTTP 201 Created; new user record created with hashed password (bcrypt).
2	POST /auth/login with valid email and password.	HTTP 200 OK; response body contains access_token (JWT string) and token_type: 'bearer'.
3	POST /auth/login with a valid email but incorrect password.	HTTP 401 Unauthorized; body contains error message 'Invalid credentials'.
4	POST /auth/login with missing password field.	HTTP 422 Unprocessable Entity; validation error for missing field.
5	GET /auth/me with the JWT returned in step 2 (Bearer token in header).	HTTP 200 OK; body returns authenticated user's id, email, and role.
6	GET /auth/me with an expired or tampered JWT token.	HTTP 401 Unauthorized; body contains 'Could not validate credentials'.
7	Verify stored password hash in DB for the registered user.	Password field is a bcrypt hash (starts with $2b$); plaintext password is not stored (NFR-08).

Post-condition	Authenticated session token stored; subsequent requests can use the token.

TC-02	Answer Submission & AI Scoring Engine	PASS
Use Case: UC-SCORE — Score Answer (AI)		*Priority: Critical*
Objective	*Ensure that submitting a text answer to POST /scoring/submit triggers the AI scoring pipeline (sentence-transformers cosine similarity) and returns a structured 4-axis score with feedback.*	
Preconditions	Authenticated user session with valid JWT. Question ID Q-042 (Binary Trees) exists in the DB with a rubric.	

#	Action / Input	Expected Result
1	POST /scoring/submit with {question_id: 'Q-042', role: 'SWE', answer_text: ''} and valid Bearer token.	HTTP 200 OK; response contains score object with four numeric fields: technical_depth, clarity, completeness, structure (each 0–100), plus feedback.
2	POST /scoring/submit with an empty answer_text field.	HTTP 422 Unprocessable Entity; validation error: 'answer_text must not be empty'.
3	POST /scoring/submit with answer_text containing only whitespace.	HTTP 400 Bad Request; error: 'Answer contains no meaningful content'.
4	POST /scoring/submit without Authorization header.	HTTP 401 Unauthorized; 'Not authenticated'.
5	POST /scoring/submit with a non-existent question_id.	HTTP 404 Not Found; 'Question not found'.

Post-condition	Score record persisted to DB linked to user_id and question_id.

TC-03	Feedback Generation During Scoring	PASS
Use Case: UC-FEEDBACK — Generate Feedback	*Priority: High*	
Objective	*Confirm that POST /scoring/submit returns deterministic rubric feedback inline with the score, that missing rubric concepts are surfaced, and that the response degrades gracefully when the Ollama AI Coach service is unavailable.*	
Preconditions	Authenticated user with valid JWT. Question Q-042 rubric has at least two identifiable concept gaps when a partial answer is submitted.	

#	Action / Input	Expected Result
1	POST /scoring/submit with a partial answer that omits at least two rubric concepts.	HTTP 200 OK; response includes a feedback array alongside the score axes.
2	Inspect the feedback field in the response from step 1.	feedback array contains at least one item per missing rubric concept, each with rubric_element and suggestion fields.
3	POST /scoring/submit with the same partial answer a second time (determinism check).	HTTP 200 OK; score axes and rubric feedback are identical to the first response (deterministic).
4	POST /scoring/submit with Ollama service stopped/mocked as unavailable.	HTTP 200 OK; scoring still succeeds; Ollama coaching fields are absent or null but the request does not fail.
5	Inspect feedback items: verify each references a rubric element and includes a suggestion.	Each feedback item's rubric_element matches a keyword from the question rubric; each suggestion is a non-empty string of at least 15 characters.

Post-condition	Feedback is stored as part of the score record and retrievable via GET /history.

TC-04	Answer History Retrieval	PASS
Use Case: UC-PROGRESS — Track Progress	*Priority: High*	
Objective	*Validate that GET /history returns the authenticated user's answer records with question prompt, score, feedback, and timestamp, and that the endpoint enforces JWT protection.*	
Preconditions	Authenticated user has at least 5 completed answer submissions across two different job roles (SWE and Data Science).	

#	Action / Input	Expected Result
1	GET /history with valid JWT (no filters).	HTTP 200 OK; returns list of answer records, newest first.
2	GET /history with valid JWT; inspect a single returned record's fields.	Each record includes: question prompt, answer text, score object (4 axes), feedback, and timestamp.
3	GET /history with valid JWT for a new user with zero submissions.	HTTP 200 OK; returns empty list [].
4	GET /history without Authorization header.	HTTP 401 Unauthorized; 'Not authenticated'.
5	GET /history with JWT belonging to User B; confirm only User B's records are returned.	HTTP 200 OK; response contains only User B's records — no cross-user data leakage.

Post-condition	No data mutation; read-only GET request.

TC-05	Role-Based Question Retrieval	PASS
Use Case: UC-QBANK — Manage Question Bank	*Priority: High*	
Objective	*Verify that GET /questions returns seeded questions correctly filtered by role, that each question contains the rubric fields required for scoring, and that invalid role values are handled gracefully.*	
Preconditions	FastAPI server running with questions seeded from backend/data/questions.json via seed_questions. At least 5 active questions exist for each role (SWE, DataScience, PM, Behavioral).	

#	Action / Input	Expected Result
1	GET /questions with no query params and valid JWT.	HTTP 200 OK; returns list of all active questions across all roles.
2	GET /questions?role=SWE with valid JWT.	HTTP 200 OK; returns only SWE-role questions.
3	GET /questions?role=DataScience with valid JWT.	HTTP 200 OK; returns only DataScience-role questions.
4	GET /questions?role=Behavioral with valid JWT.	HTTP 200 OK; returns only Behavioral-role questions.
5	GET /questions?role=INVALID_ROLE with valid JWT.	HTTP 422 Unprocessable Entity or HTTP 400; validation error for unrecognised role value.
6	Inspect a returned SWE question object for required rubric fields.	Question object contains: id, role, text, ideal_answer, concepts (list), and optionally dimension_concepts.

Post-condition	No state change; read-only endpoint. Questions are managed via seed, not admin CRUD.

TC-06	Answer & Score Database Persistence	PASS
Use Case: UC-SCORE — Score Answer (AI)	*Priority: High*	
Objective	*Verify that submitted answers and computed scores are persisted to PostgreSQL and survive a backend process restart, satisfying NFR-12.*	
Preconditions	Authenticated user with valid JWT. PostgreSQL test instance running with applied Alembic migrations.	

#	Action / Input	Expected Result
1	POST /scoring/submit with a valid answer; record the returned score_id.	HTTP 200 OK; score_id returned in response.
2	Query the DB directly: SELECT * FROM answers WHERE user_id = .	Answer row exists with correct user_id, question_id, answer_text, and created_at.
3	Query the DB directly: SELECT * FROM scores WHERE answer_id = .	Score row exists, links to the correct answer_id, and contains all four axis values.
4	Restart the FastAPI backend process/container.	Backend restarts cleanly with no migration errors.
5	GET /history with valid JWT after restart.	HTTP 200 OK; the previously submitted answer and score are still present in the history list.

Post-condition	Answer and score records persist across backend restarts (NFR-12).

TC-07	Deterministic Numeric Scoring	PASS
Use Case: UC-SCORE — Score Answer (AI)	*Priority: High*	
Objective	*Confirm that the numeric scoring pipeline produces identical 4-axis scores when the same answer is submitted against the same question rubric across repeated invocations (NFR-13).*	
Preconditions	Authenticated user with valid JWT. Question Q-042 exists with a fixed rubric. Ollama may be available or unavailable — Ollama text output is excluded from the determinism assertion.	

#	Action / Input	Expected Result
1	POST /scoring/submit with a fixed answer string for Q-042. Record all four axis values.	HTTP 200 OK; score object returned with four numeric axes.
2	Repeat POST /scoring/submit with the identical answer string 4 more times (5 total).	HTTP 200 OK for all 5 requests; no errors.
3	Compare technical_depth across all 5 responses.	All 5 technical_depth values are identical (float equality).
4	Compare clarity, completeness, and structure across all 5 responses.	All 5 values for each of clarity, completeness, and structure are identical across responses.
5	Compare rubric feedback items (rubric_element and suggestion) across all 5 responses.	Rubric feedback arrays are identical across all 5 responses; Ollama coaching text is excluded from comparison.

Post-condition	5 score records persisted in DB; numeric axes are identical across all rows for the same input.

TC-08	JWT Protection on Protected Endpoints	PASS
Use Case: UC-AUTH — Authenticate User	*Priority: Critical*	
Objective	*Verify that all protected endpoints reject requests with missing, expired, or tampered JWTs, satisfying NFR-07.*	
Preconditions	FastAPI server running. A valid JWT and a known-expired JWT are available. A tampered token is constructed by modifying the payload segment of a valid JWT.	

#	Action / Input	Expected Result
1	POST /scoring/submit without Authorization header.	HTTP 401 Unauthorized; 'Not authenticated'.
2	GET /history without Authorization header.	HTTP 401 Unauthorized; 'Not authenticated'.
3	POST /scoring/submit with a tampered JWT (payload modified).	HTTP 401 Unauthorized; 'Could not validate credentials'.
4	GET /history with a tampered JWT.	HTTP 401 Unauthorized; 'Could not validate credentials'.
5	POST /scoring/submit with an expired JWT.	HTTP 401 Unauthorized; token expiry error.
6	GET /history with an expired JWT.	HTTP 401 Unauthorized; token expiry error.

Post-condition	No data returned or mutated for any unauthenticated or invalidly authenticated request (NFR-07).

3. Test Environment & Tools

Component	Details
Framework	FastAPI (Python 3.11)
Testing Library	pytest + httpx (AsyncClient)

Auth Mechanism	JWT via python-jose
AI Scoring	sentence-transformers (all-MiniLM-L6-v2) + cosine similarity
Database	PostgreSQL (test instance with fixtures)
CI Pipeline	GitHub Actions — runs on every pull request
Coverage Target	≥ 80% line coverage for all backend modules